Unofficial Documentation of the Behringer/Midas AES50 AUX-Communication-Protocol

v0.4, 26.03.2026

More information about the OpenSource AES50 VHDL-IP Core:

https://github.com/YetAnotherElectronicsChannel/AES50

1. Technical background

Behringers X32- and WING-series as well as Midas' M32-series are using AES50 to connect multiple digital stageboxes to the audio-mixing-console – or even multiple mixing-consoles itself. AES50 (or SuperMAC) is based on 100Mbit-ethernet and uses standard ethernet-transceivers for the communication.

The bandwidth of 100Mbit/s allows the transmission of 48 audio-channels in both directions. 48 channels with 32 bit per sample and a maximum sample-rate of 48kHz results in around 74Mbit/s. The remaining 26Mbit/s are used for protocol-overhead as well as an additional auxiliary-channel.

This auxiliary-channel is used for detecting the connected device and control the headamp-gains. It has a fixed bandwidth of roughly 5 Mbit/s, but depends on the selected sample-rate, as the auxiliary data-stream is synchronous with the audio-transmission.

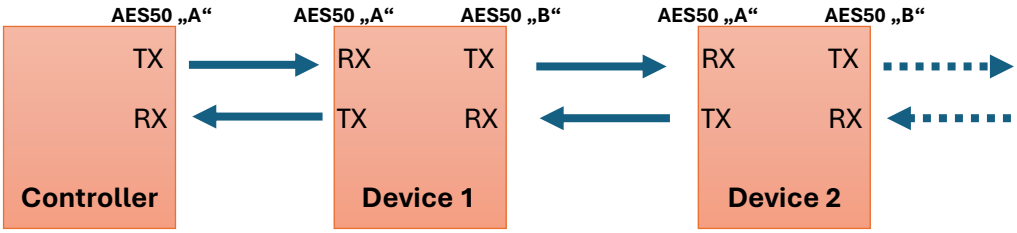
According to the AES50 specification, there was a proposal (though not mandatory) to use standard Ethernet frames that could be tunneled through this auxiliary data channel - (ever wondered about the mysterious Ethernet port on the DN9630?) -> Therefore, it can be assumed that this proposed approach using virtual Ethernet frames is also used in other AES50 devices on the market.

While reverse-engineering the protocol it turned out that Behringer is using plain byte-based communication for the payload-data. According to the AES50-specification this data is data-scrambled and bit-stuffed into the remaining bandwidth. It behaves like a regular audio-data so the OpenSource AES50-VHDL-core offers either an additional regular TDM-stream for the aux-data, or - for convenience – an easy to use UART-port with RxD and TxD.

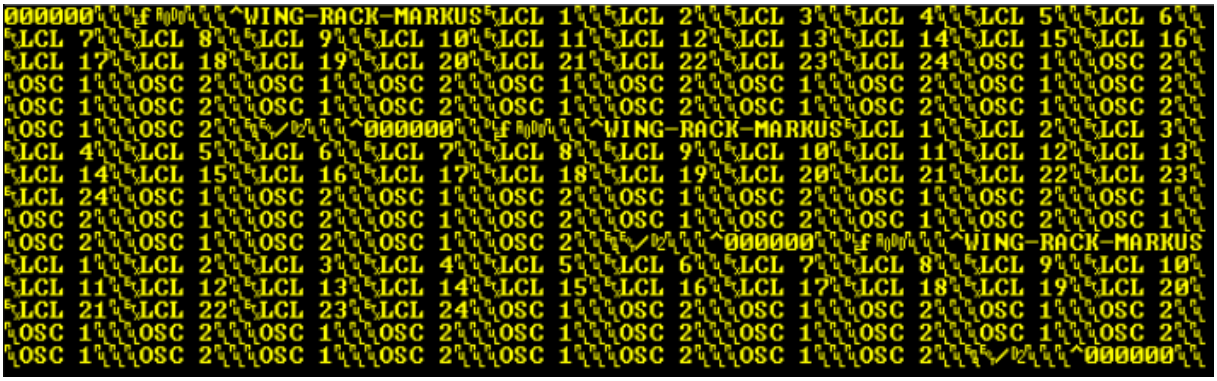
The communication is based on a simple message-protocol with a protocol-header and payload-bytes for device- and channel-naming and controlling the gain of the input-channels.

2. Messages

As AES50 uses a daisy-chaining-connection where each device collects data from the subordinated device connected to the “B”-port, merges the data with its own data and forwards it to the “A”-port. Behringer recommends not to use more than 3 devices in the chain, which reflects in some of the chosen messages below.



When connecting the auxiliary-data to a regular terminal-application, ASCII-characters can be seen:



Let’s start with the individual message. Each message starts with a 1-byte message identifier, followed by an 8-bit length-information (number of 32-bit words) and a 16-bit checksum. The following message-IDs have been found so far:

Message ID	Description
0x05	Device-Type and properties
0x10	Device- and Channelnames
0x01	Headamp-control
0x11	TBD / only seen on WING
0x12	TBD / only seen on WING

2.1. Device-Type and properties (0x05 0x04)

This short command is sent every few seconds and has the following structure:

```
1 Byte  for Message-ID
1 Byte  number of 32-bit words
2 Bytes 16-bit checksum
4 Bytes for 4-char ASCII-String (Device-Types)
6 Bytes for 6-char ASCII-String (Device-Properties)
2 Bytes 0x00 0x00 (byte-stuffing to 16 bytes)
```

Example:

□	□	□	□			^	P	2	2	0	0	0	0	␀	␀
05	04	B3	C1	20	20	5E	50	32	32	30	30	30	30	00	00

The following strings have been observed:

String	Connected Device
^000000	1x WING
P220000	1x SD16
A220000	1x S16
PA222200	1x S16 + 1x SD16
^P220000	1x SD16 + 1x WING

This string seems to tell the AES50-controller which device is connected and if a headamp-control or other options are present. The first chars are used for the connected devices:

Byte	Char	Type	Description	Byte	Char	Type	Description
65	A	S16	S16 Stagebox	86	V	DL231	DL231
66	B	X32C	X32 Compact	87	W	S32	S32 Stagebox
67	C	X32	X32 Fullsize	88	X	S32 B	S32 Stagebox
68	D	25X B	DL251	89	Y	DL32	DL32 Stagebox
69	E	DL25X	DL251HA	90	Z	DL32B	DL32 Stagebox
70	F	S16 B	S16 Stagebox	91	[	WING	WING Fullsize
71	G	Z32	Z32	92	\		WING Compact
72	H	T8	T8	93	}		WING Compact
73	I	X32P	X32 Producer	94	^		WING Rack
74	J	X32rck	X32 Rack	95	_		WING Full Black
75	K	X32c	X32 Core	96	`		???
76	L	M32	M32 Fullsize	97	a	M23C	M32 Compact
77	M	M32R	M32 Rack	98	b	DP48	DP48
78	N	DL16	DL16 Stagebox	99	c	HUB4	HUB4
79	O	DL16B	DL16 Stagebox	100	d	DL8	DL8 Stagebox
80	P	SD16	SD16 Stagebox	101	e	DL8 B	DL8 Stagebox
81	Q	SD16B	SD16 Stagebox	102	f		???
82	R	SD8	SD8 Stagebox	103	g		P24 Hub
83	S	SD8B	SD8 Stagebox	104	h		P24 Hub
84	T	15X B	DL 15X	105	i		WING Compact(?)
85	U	DL15X	DL 15XHA				

Next to the product-type 6 chars seem to define the type of headamp-control:

Headamp-Chars	Product
000000	No headamp-control on all three devices
220000	First device has 16 gain-controls
222200	First two devices have 16+16 gain-controls

C-Example

```
void sendDeviceTypeAndProperty() {
      data[0] = 0x05;
      data[1] = (16 / 4); // message-length in 32-bit words
      data[2] = 0; // checksum
      data[3] = 0; // checksum

      // AES50-DeviceChars for the last four connected devices
      // (AES50 seems to support up to 6, but only the last 4 are transmitted)
      data[4] = 0;   // fourth device in AES50-chain
      data[5] = 0;   // third device in AES50-chain
      data[6] = 0;   // second device in AES50-chain
      data[7] = 'C'; // first device in AES50-chain: X32

      // data[8 .. 13] <- '0' if no headamp-control, '2' if headamp-
      // control is present
      data[8] = '0';  // AES50 Ch 1-8 have no headamp-control in current device
      data[9] = '0';  // AES50 Ch 9-16 have no headamp-control in current device
      data[10] = '0'; // AES50 Ch 17-24 have no headamp-control in current device
      data[11] = '0'; // AES50 Ch 25-32 have no headamp-control in current device
      data[12] = '0'; // AES50 Ch 33-40 have no headamp-control in current device
      data[13] = '0'; // AES50 Ch 41-48 have no headamp-control in current device

      // byte-stuffing to have full divider by 4
      data[14] = 0x00;
      data[15] = 0x00;

      // calculate and add checksum to begin of message
      addChecksum(&data[0]);

      // send data over UART to AES50 IP-core
      Serial1.write(&data[0], 16);
}
```

2.2. Device- and Channelnames (0x10 0x66)

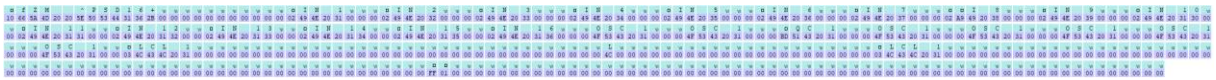
Every second a 408 byte-message is published from each device to its “A”-port to the upstream-device. As the data of the subordinated devices are merged with the data of the current device, every message contains fixed information of all 48 channels plus the device-name of the current device. If additional devices are connected to the “B”-port, a “+” is added to the device name:

```
1 Byte  for Message-ID
1 Byte  number of 32-bit words
2 Bytes 16-bit checksum
4 Bytes for 4-char ASCII-String (Device-Types)
16 Bytes for ASCII-Device-String
1 Byte for separation with fixed 0x02
384 Bytes for 48 7-char-ASCII-Ch-Names (each with 0x00 or 0x02-termination)
```

Here an example for a connected SD16 with a WING connected to its “B”-port:

```
 ◘  f  Z  M                ^  P  S  D  1  6  +  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ␀  ◘  I  N     1  ␀  ␀  ␀  ◘  I  N     2  ␀  ␀  ␀  ◘  I  N     3  ␀  ␀  ␀  ◘
10 66 5A 4D 20 20 5E 50 53 44 31 36 2B 00 00 00 00 00 00 00 00 00 00 00 00 02 49 4E 20 31 00 00 00 02 49 4E 20 32 00 00 00 02 49 4E 20 33 00 00 00 02
```

And the full message as picture:



And in ASCII with special-chars:

```
<16>fZM  ^PSD16+<\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><2>IN 1<\0><\0><\0><2>IN
2<\0><\0><\0><2>IN 3<\0><\0><\0><2>IN 4<\0><\0><\0><2>IN 5<\0><\0><\0><2>IN 6<\0><\0><\0><2>IN
7<\0><\0><\0><2>©I 8<\0><\0><\0><2>IN 9<\0><\0><\0><2>IN 10<\0><\0><2>IN 11<\0><\0><2>IN
12<\0><\0><2>IN 13<\0><\0><2>IN 14<\0><\0><2>IN 15<\0><\0><2>IN 16<\0><\0><\0>OSC
1<\0><\0><\0>OSC 1<\0><\0><\0>OSC 1<\0><\0><\0>OSC 1<\0><\0><\0>OSC 1<\0><\0><\0>OSC
1<\0><\0><\0>OSC 1<\0><\0><\0>OSC 1<\0><\0><3>LCL
1<\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><
\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0>L<\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\
0><\0><\0><\0><\0><\0><\0><\0><\0><\0><3>LCL
1<\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><
\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0
><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0>ÿ<1><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><
\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0
><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><\0><
\0><\0><\0><\0>
```

Example in C

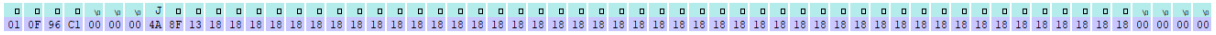
```c
void sendNames() {
	data[0] = 0x10;
	data[1] = (408 / 4); // message-length in 32-bit words
	data[2] = 0; // checksum
	data[3] = 0; // checksum

	// AES50-DeviceChars for the last four connected devices
	// (AES50 seems to support up to 6, but only the last 4 are transmitted)
	data[4] = 0;
	data[5] = 0;
	data[6] = 0;
	data[7] = 'C'; // X32

	// the next bytes are zero-terminated ASCII-strings.
	// First clear the data-array with zeros
	for (int i = 8; i < 408; i++) {
		data[i] = 0x00;
	}

	// insert zero-terminated device-string of current AES50-device
	sprintf((char*)&data[8], "X32");
	data[24] = 0x02; // add a 0x02 as "START OF TEXT"

	// from data[25] 48x 7-char ASCII-Name for each channel is
	// sent followed by a 0x02
	for (int i = 0; i < 48; i++) {
		sprintf((char*)&data[25 + (i * 8)], "IN % u", i + 1);
		data[25 + (i * 8) + 7] = 0x02;
	}
	data[407] = 0x0A; // add a linefeed at the end

	// calculate and add checksum to begin of message
	addChecksum(&data[0]);

	// send data over UART to AES50 IP-core
	Serial1.write(&data[0], 408);
}
```

2.3. Headamp-Control (0x01 0x0F)

Every few seconds the headamp-gain-control is transmitted – or by manually changing these values. The first part of this message seems to be redundant to the device-type message followed by the settings of the headamps:

```
1 Byte  for Message-ID
1 Byte  number of 32-bit words
2 Bytes 16-bit checksum
4 Bytes for 4-char ASCII-String (Device-Types)
48 Bytes for headamp-control
4 Bytes 0x00 0x00 0x00 0x00 (byte-stuffing to 60 bytes)
```

Example:


01 0F 96 C1 00 00 00 4A 8F 13 18 18 18

Example in C

```c
void sendHeadampMessage() {
	data[0] = 0x01;
	data[1] = (60 / 4); // message-length in 32-bit words
	data[2] = 0; // checksum
	data[3] = 0; // checksum

	// AES50-DeviceChars for the last four connected devices
	// (AES50 seems to support up to 6, but only the last 4 are transmitted)
	data[4] = 0;
	data[5] = 0;
	data[6] = 0;
	data[7] = 'C'; // X32

	// now insert the headamp-gains
	// the specific value depends on the connected AES50 device as some
	// devices have more gain-options than others
	//
	// An S16 has settings between -2.0dB and 45.5dB resulting in 47.5dB range
	// with 2.5dB steps -> 47.5 / 2.5 = 19. So we have settings between
	// 0 = -2.0dB and 19 = 45.5dB
	for (int i = 0; i < 48; i++) {
		data[8 + i] = (uint8_t) roundf(gainMap(headAmpGain[i], -2.0f, 45.5f, 0, 19));

		if (headAmpPhantom[i]) {
			data[8 + i] |= 0x80;
		}
	}

	// a single 32-bit word with zeros for finalizing the message
	data[56] = 0;
	data[57] = 0;
	data[58] = 0;
	data[59] = 0;

	// calculate and add checksum to begin of message
	addChecksum(&data[0]);

	// send data over UART to AES50 IP-core
	Serial1.write(&data[0], 60);
}
```

2.4. Calculating the Checksum-Word

The checksum is calculated based on the given AES50-device-char of the current device as a seed and then XOR'ed 16-bit words of the payload with 0xFFFF. The 16-bit checksum is then distributed as 8-bit values to buffer-index 2 and 3 (counted from 0).

Example in C

```
void addChecksum(uint8_t* buf) {
	uint8_t messageID = buf[0];
	uint8_t len = buf[1];
	uint8_t deviceChar = buf[7];

	// initializing
	uint32_t ids4 = (uint32_t)deviceChar << 24;
	uint16_t cks = (uint16_t)(ids4 >> 16) ^ (uint16_t)(ids4 & 0xFFFF);

	uint32_t* dataWords = (uint32_t*)&buf[8];

	if (len > 2) {
		for (int i = 0; i <= (len - 3); i++) {
			uint32_t currentWord = dataWords[i];
			cks -= (uint16_t)(currentWord >> 16) ^ (uint16_t)(currentWord & 0xFFFF);
		}
	}
	uint16_t finalCks = cks ^ 0xFFFF;

	buf[2] = (uint8_t)(finalCks & 0xFF);
	buf[3] = (uint8_t)(finalCks >> 8);
}
```